

Jones High Level Design Document

Predictive Modeling for Bitcoin Volatility Forecasting High Level Design Document

Patrice Jones
CSMS Degree Program
Full Sail University
April 2026

Table of Contents

Introduction.....	3
Project Concept / Pitch.....	3
Technical Design Goal(s)	3
Development APIs / Programming Languages.....	3
Development Standards / Cold-Start Procedures.....	3
Project Functionality.....	5
User Modes	5
Main Mechanics / Sub Mechanics	5
AI Module(s).....	5
User Interface / Inputs / Sensors	5
Graphics / Animation / Video	6
Audio.....	6
Networking	6
Databases / Server-Side Technology	6
Localization Plan	6
Custom Topics	6
Additional Technical Information.....	6
Mockups / Flowcharts.....	6
Production Documentation	8
Feature List	8
Master Task List	9
Development Roadmap.....	11
Verification and Validation (V&V).....	11
Quality Test List	11
References.....	13

Introduction

This project is a context-aware Bitcoin volatility forecasting system designed to support short-horizon risk analysis. The system loads daily Bitcoin market data, transforms it into volatility-oriented forecasting inputs, optionally extends those inputs with contextual variables such as sentiment, investor attention, and uncertainty-related signals, and compares multiple forecasting approaches under the same chronological evaluation design. The results are then organized into rankings, error summaries, visual comparisons, and exportable outputs.

The project begins from a practical problem: price movement alone does not tell a user whether market conditions are becoming more unstable. Volatility matters because it shapes uncertainty, risk exposure, and the reliability of short-term interpretation. For that reason, the system forecasts volatility directly rather than treating it as a side effect of price prediction. The central design question is whether context-aware modeling provides more useful short-horizon Bitcoin risk insight than a price-only workflow, and whether that difference can be shown through a consistent evaluation structure.

Project Concept / Pitch

A forecasting system that compares price-only and context-aware models to reveal when Bitcoin volatility is becoming harder to ignore.

Technical Design Goal(s)

The main technical goal is to build a forecasting workflow that estimates short-horizon Bitcoin volatility in a way that remains disciplined, modular, and interpretable. The project is not trying to prove that the most flexible model always wins. It is trying to determine whether a context-aware feature space improves predictive performance over a price-only workflow when both are evaluated under the same conditions.

A second goal is to preserve chronology across the full pipeline. In forecasting, leakage can make a model appear more accurate than it really is. That risk affects preprocessing, scaling, target construction, and sequence generation, so chronological integrity is treated as a design requirement rather than a minor implementation detail.

A third goal is to create a result layer that supports comparison rather than simply displaying output. The project should make it easy to inspect model rankings, error metrics, actual-versus-predicted behavior, and the effect of contextual variables. The system is strongest when those differences are visible and testable.

Development APIs / Programming Languages

- Python 3.11+ for implementation
- pandas for data loading, preprocessing, and tabular operations
- NumPy for numerical transformations
- scikit-learn for Random Forest modeling and related utilities
- PyTorch for LSTM implementation and training
- Matplotlib or Plotly for charts and visual outputs
- Streamlit or Flask for the dashboard and interface layer
- Git / GitHub for version control and collaboration

Development Standards / Cold-Start Procedures

The project is organized so preprocessing, contextual integration, modeling, evaluation, and presentation remain separate. Configuration values should stay outside the core business logic where possible. Descriptive names should be used instead of compressed abbreviations, and assumptions should be documented directly in code comments and technical notes.

Chronology must be preserved across the full workflow. No feature, scaling step, or sequence-building step should allow future information to enter training or feature construction in a way that weakens the validity of the forecast. Fixed seeds should be used where stochastic behavior could affect reproducibility. Intermediate and final artifacts should be stored in structured output folders instead of being scattered across the project root.

Repository Structure

```
project/
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── preprocessing.py
│   ├── context_features.py
│   ├── baseline_models.py
│   ├── rf_model.py
│   ├── lstm_model.py
│   ├── metrics.py
│   └── dashboard.py
├── outputs/
│   ├── charts/
│   ├── tables/
│   └── exports/
├── requirements.txt
├── main.py
└── README.md
```

Cold-Start Procedure

1. Clone the repository.
2. Open the project folder.
3. Create and activate a virtual environment.
4. Install dependencies from requirements.txt.
5. Confirm the expected Python version.
6. Place the Bitcoin dataset in the raw data folder.
7. Run preprocessing to verify engineered feature creation.
8. Run one baseline model to confirm metrics and output storage.
9. Launch the dashboard to verify result rendering.

Example commands:

```
git clone <repo-url>
cd project-folder
Python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python main.py
.venv\Scripts\activate # Windows
```

Project Functionality

User Modes

The application is designed primarily as a single-user analytical system. The intended user is a researcher, technically informed analyst, or reviewer who wants to load a dataset, generate features, run forecasting models, and compare results.

- Price-only forecasting mode using only OHLCV-derived features such as returns, realized volatility, and EWMA-related inputs.

- Context-aware forecasting mode that extends the base workflow with aligned external variables such as sentiment, investor attention, or uncertainty-sensitive measures.
- Review / export mode that allows the user to inspect rankings, charts, metric summaries, and export results for later reporting.

Main Mechanics / Sub Mechanics

The core mechanics of the project are analytical rather than game-like, but they still represent the system behaviors the user depends on.

Main mechanics include importing Bitcoin market data, validating file structure and chronology, generating volatility-oriented features, optionally integrating contextual variables, running forecasting models, comparing model performance, and reviewing or exporting results.

Sub mechanics include duplicate and missing-value checks, rolling realized volatility calculation, EWMA feature generation, forward target construction, contextual date alignment, chronological train-test splitting, sequence generation for LSTM, metric calculation and ranking, and optional risk flagging.

AI Module(s)

The project includes machine learning modules as part of the forecasting pipeline.

- Random Forest for tabular nonlinear regression
- LSTM for sequence-based volatility forecasting

These models are not included simply to increase model variety. They are included because they allow direct comparison against simpler baselines such as Historical Volatility and EWMA. That comparison shows whether more flexible modeling actually adds predictive value under the same conditions.

User Interface / Inputs / Sensors

The interface functions as a compact analytical dashboard rather than a broad trading interface. The user should be able to load a dataset, confirm preprocessing status, choose one or more models, toggle contextual inputs, run forecasts, and inspect results in one readable workflow.

- dataset upload or selection
- validation feedback messages
- feature review summary
- model selection controls
- context on/off control
- results dashboard with charts and ranking table
- export button

No special sensors or peripherals are required. The application uses standard desktop input such as keyboard and mouse interaction.

Graphics / Animation / Video

The project does not rely on animation or video as a core system component. Graphics are limited to analytical visuals such as line charts for actual versus predicted volatility, metric summary tables, ranking displays, and optional alert indicators. These visuals will be generated through Matplotlib or Plotly and rendered inside the dashboard.

Audio

Audio is not part of the system. The project does not require sound effects, background music, voice acting, or interactive audio behavior.

Networking

The current version of the project does not require networking as a core feature. The application is not being built as a distributed platform, real-time service, or multiplayer environment. It does not need packet design, synchronization logic, or remote reliability handling in the current capstone scope.

Databases / Server-Side Technology

The project does not require a full database or server-side architecture to function in its current form. The expected workflow uses local CSV input for the Bitcoin dataset, optional local structured files for contextual variables, and local output files for charts, tables, and exports.

Localization Plan

There is no localization plan for the current implementation. The system is intended for English-language academic and technical use, and the amount of interface text does not justify multilingual support in this version.

Custom Topics

The most important custom topic not fully covered by the standard sections is contextual feature integration. This is a major part of the project because it expands the system beyond price-only forecasting and directly supports the central comparative question. The conceptual role is clear, but the final contextual variables may still evolve based on data quality, overlap, and implementation feasibility.

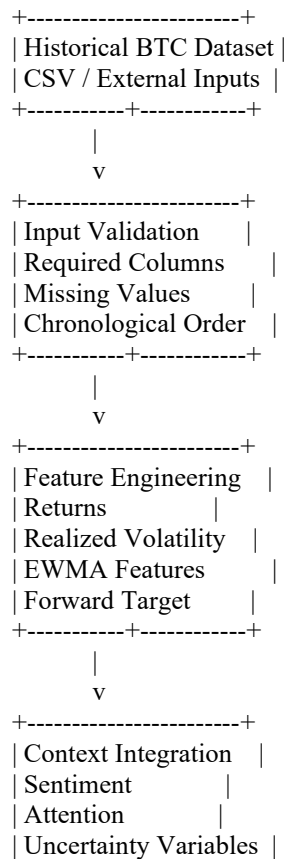
Additional Technical Information

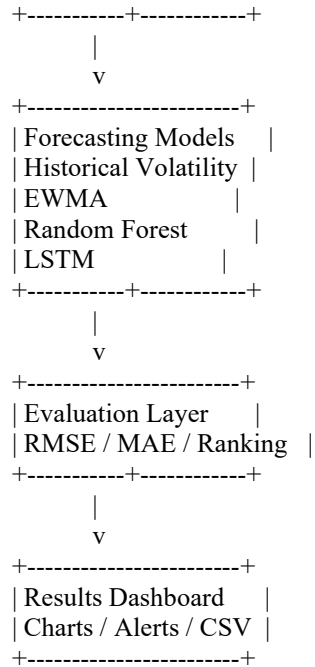
The project favors methodological credibility over unnecessary complexity. A clean baseline with consistent evaluation is more valuable than an oversized model list. A carefully aligned contextual signal is more useful than several external series that were merged without enough scrutiny. A readable result summary is more valuable than a crowded interface that hides the actual comparison.

The system should make it easy to answer a direct chain of questions: what data entered the workflow, what target and features were created, what contextual variables were included, which models were run, how they ranked, whether context improved the result, and what evidence supports that conclusion.

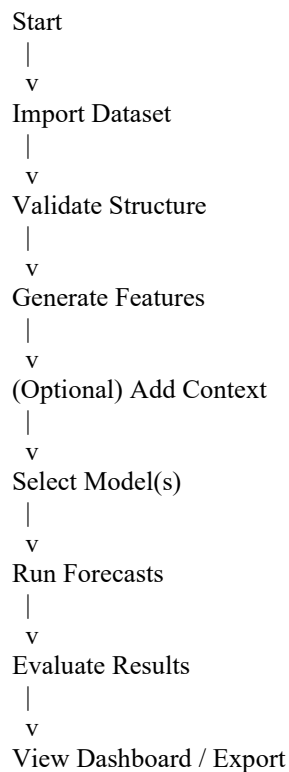
Mockups / Flowcharts

High-Level Data Flow

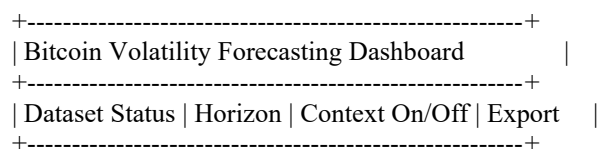




Interface Flow



Dashboard Mockup



Model Ranking Table	
Historical Volatility	RMSE MAE Rank
EWMA	RMSE MAE Rank
Random Forest	RMSE MAE Rank
LSTM	RMSE MAE Rank
+-----+	
Actual vs Predicted Volatility Chart	
+-----+	
Risk Flags Model Notes Context Comparison Summary	
+-----+	

Production Documentation

Feature List

Number	Feature	Category	Priority	Difficulty
F1	Bitcoin dataset import and validation	System	High	Medium
F2	Volatility feature engineering	System	High	Hard
F3	Contextual feature integration	System	High	Hard
F4	Historical Volatility baseline	System	High	Medium
F5	EWMA baseline	System	High	Medium
F6	Random Forest forecasting	System	High	Medium
F7	LSTM forecasting	System	High	Hard
F8	Metrics computation and ranking	System	High	Medium
F9	Results dashboard	UI	High	Medium
F10	Actual vs predicted chart view	UI	High	Medium
F11	CSV export	UI	Medium	Easy
F12	Forecast horizon selection	UI	Medium	Medium
F13	Model configuration controls	UI	Medium	Medium
F14	Volatility alert flagging	UI	Low	Easy

F15	Model explanation panel	UI	Low	Easy
F16	Python 3.11+	Tool	High	Medium
F17	pandas / NumPy	Tool	High	Medium
F18	scikit-learn	Tool	High	Medium
F19	PyTorch	Tool	High	Hard
F20	Streamlit or Flask	Tool	High	Medium
F21	Git / GitHub	Tool	High	Easy

Master Task List

Number	Task	Feature	Dependencies
T1	Set up repository structure and dependency file	F16-F21	NA
T2	Implement Bitcoin CSV loading and structural validation	F1	T1
T3	Implement chronology checks and duplicate handling	F1	T2
T4	Implement return calculation and realized volatility logic	F2	T3
T5	Implement EWMA-related feature generation and forward target	F2	T4
T6	Create processed dataset storage workflow	F2	T5
T7	Implement Historical Volatility baseline	F4	T6
T8	Implement EWMA baseline forecasting	F5	T6
T9	Build chronological split utility and evaluation module	F8	T6

T10	Implement contextual data loading and alignment rules	F3	T6
T11	Merge contextual variables with engineered dataset	F3	T10
T12	Implement Random Forest training and inference	F6	T9, T11
T13	Implement LSTM sequence generation and scaling logic	F7	T9, T11
T14	Implement LSTM training, validation, and prediction routines	F7	T13
T15	Build dashboard layout and model ranking table	F9	T9, T12, T14
T16	Add actual-vs-predicted chart view	F10	T15
T17	Add CSV export support	F11	T15
T18	Add horizon and configuration controls	F12, F13	T15
T19	Add alert logic and model explanation panel	F14, F15	T15

Development Roadmap

Milestone 1

- T1 - Set up repository structure, requirements file, and output directories.
- T2 - Implement Bitcoin CSV loading and required-column validation.
- T3 - Add chronology checks, duplicate handling, and invalid input detection.

Milestone 2

- T4 - Implement return calculation and rolling realized volatility.
- T5 - Implement EWMA-related feature generation and forward target logic.
- T6 - Save processed dataset outputs and verify target alignment.

Milestone 3

- T7 - Implement Historical Volatility baseline.
- T8 - Implement EWMA baseline forecasting.
- T9 - Build RMSE / MAE evaluation module and chronological split utility.

Milestone 4

- T10 - Implement contextual data loading and date alignment.
- T11 - Merge contextual variables into engineered dataset and document overlap rules.

Milestone 5

- T12 - Implement Random Forest training and inference pipeline.
- T13 - Build LSTM sequence generation and scaling workflow.
- T14 - Implement LSTM training, validation, and prediction routines.

Milestone 6

- T15 - Build dashboard layout and model ranking table.
- T16 - Add actual-vs-predicted chart view.
- T17 - Add CSV export support.
- T18 - Add supported horizon and model configuration controls.
- T19 - Add risk alert logic and explanation panel.

The roadmap is sequenced to protect the project's value if later features have to be reduced. As long as ingestion, preprocessing, baseline comparison, and evaluation are implemented cleanly, the system still retains its main contribution.

Verification and Validation (V&V)

Verification addresses whether a feature has been built correctly from a system perspective. Validation addresses whether the feature behaves in a way that is useful and understandable to the intended user.

Quality Test List

Task / Feature	Verification	Validation
F1 - Bitcoin dataset import and validation	Valid CSV files load correctly; invalid files return structured error messages; required columns and chronological order are checked consistently.	User can load a supported dataset and clearly understand whether it was accepted or rejected.
F2 - Volatility feature engineering	Returns, realized volatility, EWMA-related features, and forward targets are computed consistently across repeated runs.	User can move into forecasting with confidence that the engineered dataset reflects the intended volatility task.
F3 - Contextual feature integration	External variables align correctly by date; merge rules are applied consistently; incomplete overlap is flagged.	User can tell whether contextual variables were included, rejected, or only partially available.
F4 / F5 - Baseline forecasting	Historical Volatility and EWMA produce predictions for all supported test cases and save results correctly.	User receives baseline outputs that are easy to compare against later ML results.
F6 - Random Forest forecasting	Model trains and predicts on chronological train-test data without feature-target mismatch.	User can run the model and receive error metrics and saved predictions without ambiguity.

F7 - LSTM forecasting	Sequence generation, scaling, training, and prediction preserve chronology and avoid leakage.	User can compare LSTM output to the other models under the same review structure.
F8 - Metrics computation and ranking	RMSE and MAE are computed accurately and rankings update correctly across completed models.	User can identify which model performed best without doing manual calculations.
F9 / F10 - Results dashboard and chart view	Dashboard values and chart data match stored outputs exactly.	User can inspect model rankings and prediction behavior in one readable screen.
F11 - CSV export	Exported files open successfully and contain all required fields.	User can save and revisit results later without losing important context.
F12 / F13 - Horizon and configuration controls	Supported settings update the workflow correctly and invalid settings are blocked.	User can make supported adjustments without breaking the system.
F14 - Volatility alert flagging	Alerts trigger only when the selected threshold is exceeded.	User can identify elevated forecast risk quickly and clearly.
F15 - Model explanation panel	Explanation text always matches the selected model.	User can understand the basic role and limitation of each model.

References

- [1] Y. Wang, G. Andreeva, and B. Martin-Barragan, "Machine learning approaches to forecasting cryptocurrency volatility: Considering internal and external determinants," *International Review of Financial Analysis*, vol. 90, art. 102914, Nov. 2023, doi: 10.1016/j.irfa.2023.102914.
- [2] G. Dudek, P. Fiszeder, P. Kobus, and W. Orzeszko, "Forecasting cryptocurrencies volatility using statistical and machine learning methods: A comparative study," *Applied Soft Computing*, vol. 151, art. 111132, Jan. 2024, doi: 10.1016/j.asoc.2023.111132.
- [3] Z.-C. Huang, I. Sangiorgi, and A. Urquhart, "Forecasting Bitcoin volatility using machine learning techniques," *Journal of International Financial Markets, Institutions and Money*, vol. 97, art. 102064, Dec. 2024, doi: 10.1016/j.intfin.2024.102064.
- [4] Y. Xia, C. Sang, L. He, and Z. Wang, "The role of uncertainty index in forecasting volatility of Bitcoin: Fresh evidence from GARCH-MIDAS approach," *Finance Research Letters*, vol. 52, art. 103391, Mar. 2023, doi: 10.1016/j.frl.2022.103391.
- [5] S. Aras, M. O. Ozdemir, and C. Cilgin, "Uncertainty or investor attention: Which has more impact on Bitcoin volatility?" *Research in International Business and Finance*, vol. 77, pt. B, art. 103002, May 2025, doi: 10.1016/j.ribaf.2025.103002.
- [6] Kaggle, "Bitcoin Historical Data," [Online]. Available: <https://www.kaggle.com/datasets/mczielinski/bitcoin-historical-data>. [Accessed: Apr. 2026].